Numerical skills for neuroscience

Part 2 · Vectors, matrices and PCA

Matteo Lisi

Linear algebra

Tool	What it lets us do
Vector	Collect measurements into one object
Matrix	Organise data or combine measurements
Projection	Describe an observation along a chosen direction
PCA	Find directions that capture the most variation

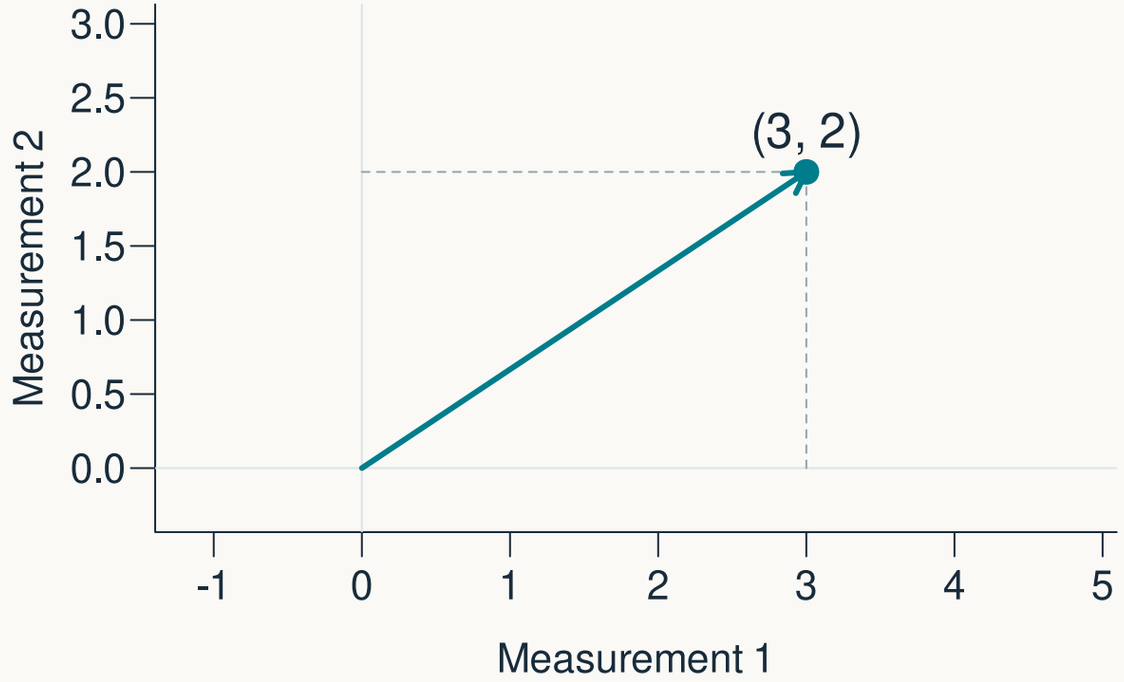
A data matrix has rows and columns

	Neuron 1	Neuron 2	Neuron 3
Trial 1	4	8	2
Trial 2	6	9	4
Trial 3	3	6	1
Trial 4	7	10	5

← One row: one observation

$$\mathbf{X} \in \mathbb{R}^{n \times p} \qquad n \text{ observations}, \; p \text{ variables}$$

A vector is a list and a point



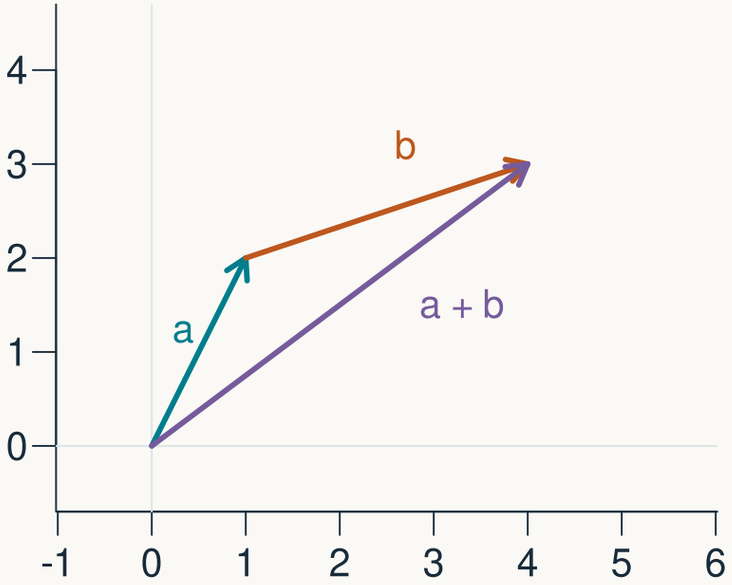
$$\mathbf{x} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

Two measurements → one point in **two dimensions**.

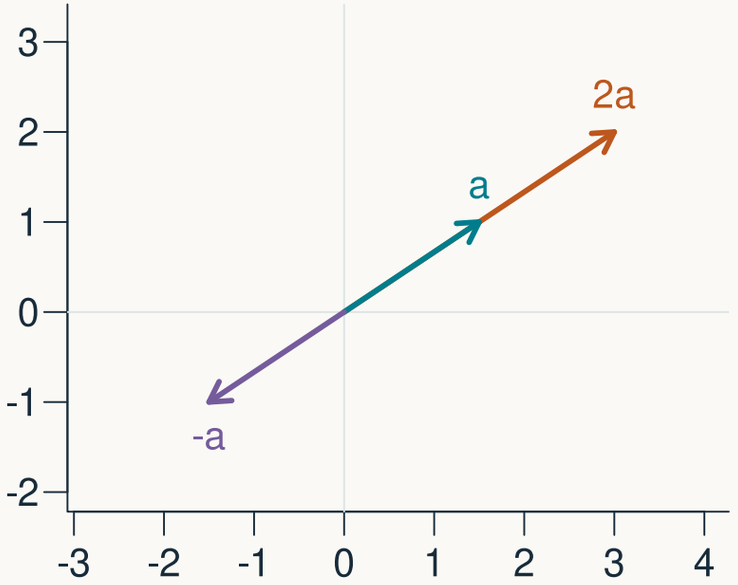
An arrow from the origin gives the same coordinates.

Add vectors; multiply them by a scalar

Add: head to tail



Scale: length and possibly direction



$$\begin{bmatrix}1\\2\end{bmatrix} + \begin{bmatrix}3\\1\end{bmatrix} = \begin{bmatrix}4\\3\end{bmatrix} \qquad 2\begin{bmatrix}1.5\\1\end{bmatrix} = \begin{bmatrix}3\\2\end{bmatrix}$$

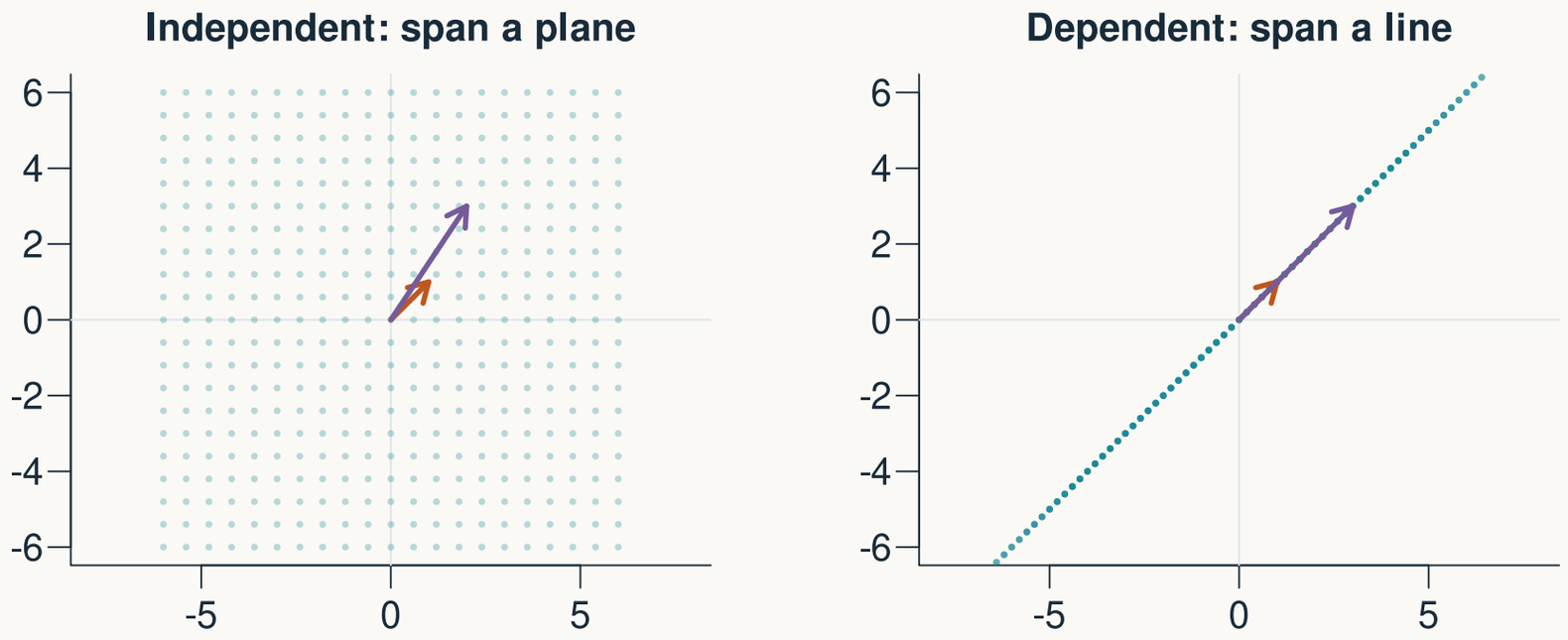
A linear combination is a weighted sum

$$c\mathbf{v} + d\mathbf{w} = c\begin{bmatrix}1\\1\end{bmatrix} + d\begin{bmatrix}2\\3\end{bmatrix} = \begin{bmatrix}c+2d\\c+3d\end{bmatrix}$$

Set $c = 2$ and $d = -1$. Where do you land?

$$2\mathbf{v} - \mathbf{w} = \begin{bmatrix}0\\-1\end{bmatrix}$$

Span: everywhere those combinations can reach



A **basis** is an independent set that spans the space being described.

A matrix packages the columns we want to combine

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} \mathbf{v} & \mathbf{w} \end{bmatrix}$$

$$\underbrace{\mathbf{A}}_{2\times 2} \underbrace{\begin{bmatrix} c \\ d \end{bmatrix}}_{2\times 1} = c\mathbf{v} + d\mathbf{w}$$

> Matrix × vector = a weighted sum of the matrix's **columns**.

A dot product multiplies, then adds

$$\mathbf{a}^\mathsf{T}\mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_p b_p$$

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = 1 \times 3 + 2 \times 1 = 5$$

> The result is **one number**.

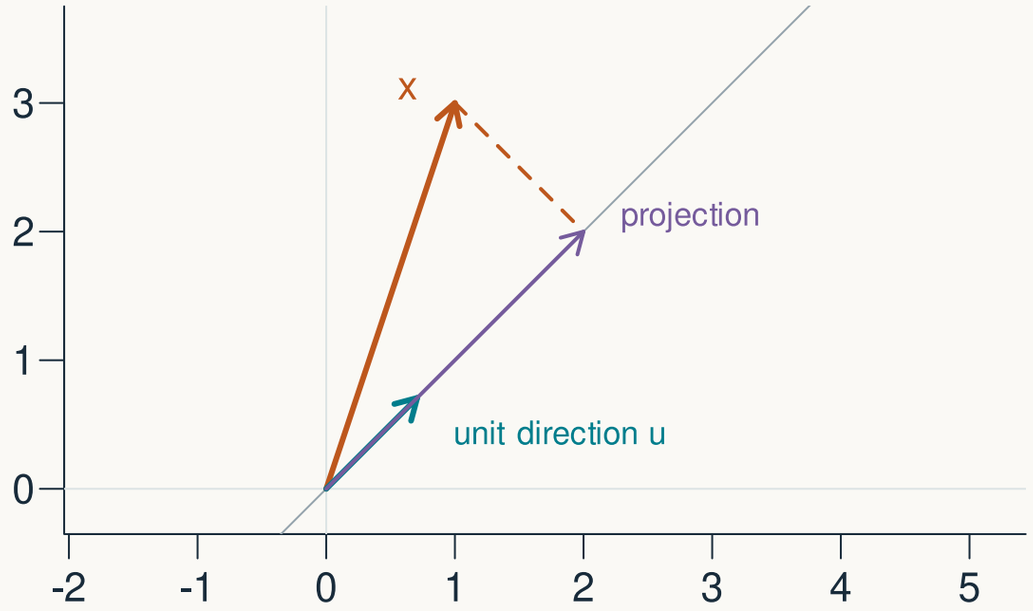
Dot products also describe alignment



$$\mathbf{a}^\mathsf{T}\mathbf{b} = \|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta$$

In 2D: $\|\mathbf{a}\| = \sqrt{a_1^2 + a_2^2}$. The angle requires **nonzero** vectors.

Projection gives a coordinate along an axis



For a **unit** direction $\mathbf{u}$:

$$z = \mathbf{x}^\mathsf{T}\mathbf{u}$$

$$\text{projected vector} = z\mathbf{u}$$

One number locates the point along this line.

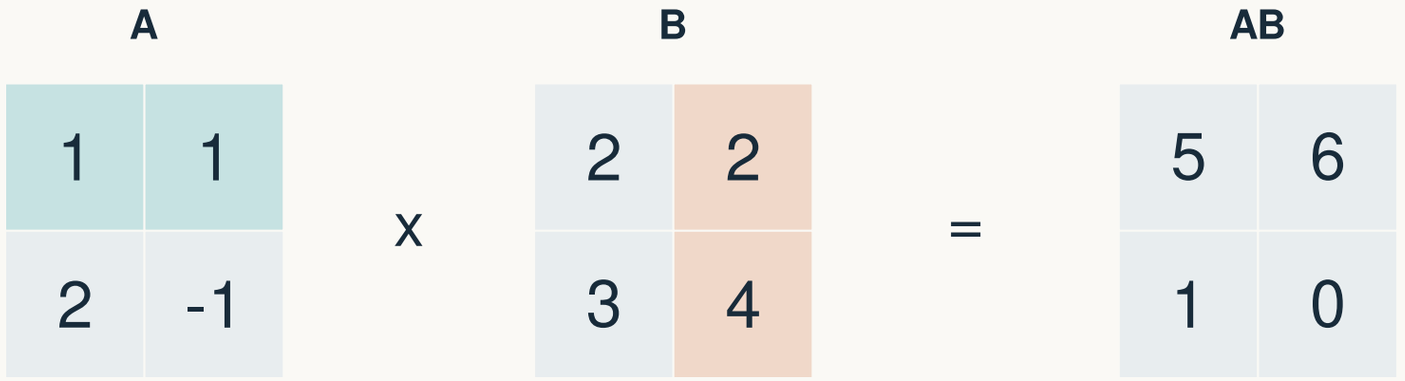
Transpose swaps rows and columns

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \Longrightarrow \quad \mathbf{A}^{\mathsf{T}} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

$$2 \times 3 \quad \longrightarrow \quad 3 \times 2$$

In R: `t(A)`.

Matrix multiplication: a row meets a column



Row 1 of A . column 2 of B: 1 x 2 + 1 x 4 = 6

The same product, one column at a time

$$\mathbf{AB} = \begin{bmatrix} \mathbf{Ab}_1 & \mathbf{Ab}_2 \end{bmatrix}$$

$$\mathbf{A}\begin{bmatrix} 2 \\ 3 \end{bmatrix} = 2\begin{bmatrix} 1 \\ 2 \end{bmatrix} + 3\begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

> Each column of the product is a combination of columns of $\mathbf{A}$.

Check the dimensions before multiplying

$$\underbrace{\mathbf{A}}_{m\times n}\underbrace{\mathbf{B}}_{n\times p} = \underbrace{\mathbf{C}}_{m\times p}$$

> $(100 \times 3)(3 \times 2) \rightarrow$ what shape?
>
> Can we reverse the order?

100×2. Reversed: inner dimensions 2 and 100 do not match.

In R, * and %*% do different jobs

Element by element

```r
A <- matrix(c(1, 1, 2, -1), 2, byrow = TRUE)
B <- matrix(c(2, 2, 3, 4), 2, byrow = TRUE)
A * B
```

```
     [,1] [,2]
[1,]    2    2
[2,]    6   -4
```

Matrix multiplication

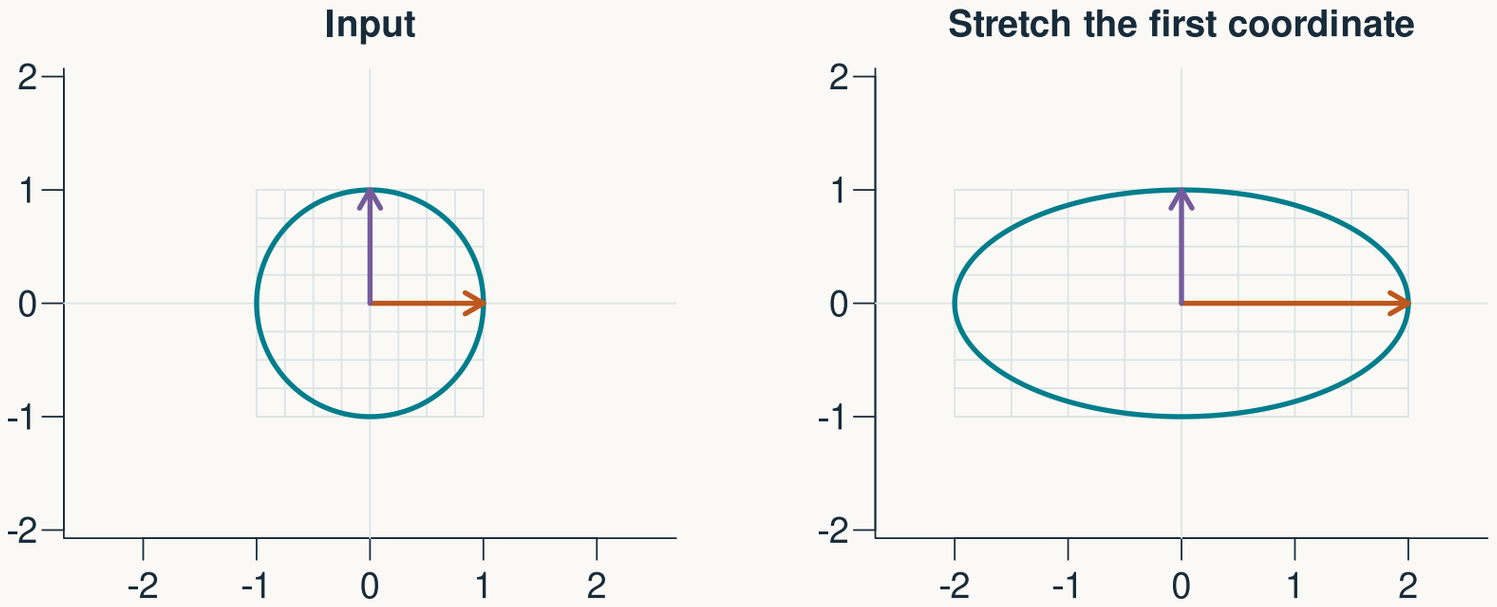
```r
A %*% B
```

```
     [,1] [,2]
[1,]    5    6
[2,]    1    0
```

Rows meet columns; products are added.

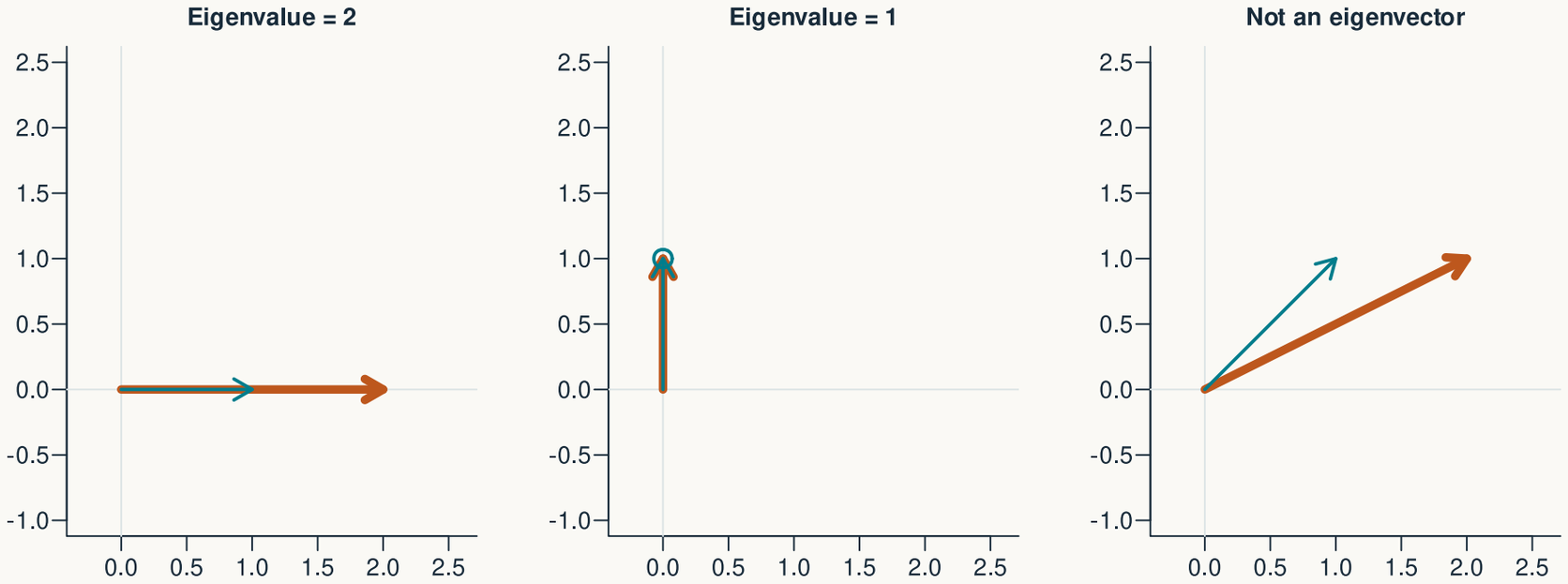
A matrix can transform a whole space

$$\mathbf{S} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \qquad \mathbf{S} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2x_1 \\ x_2 \end{bmatrix}$$



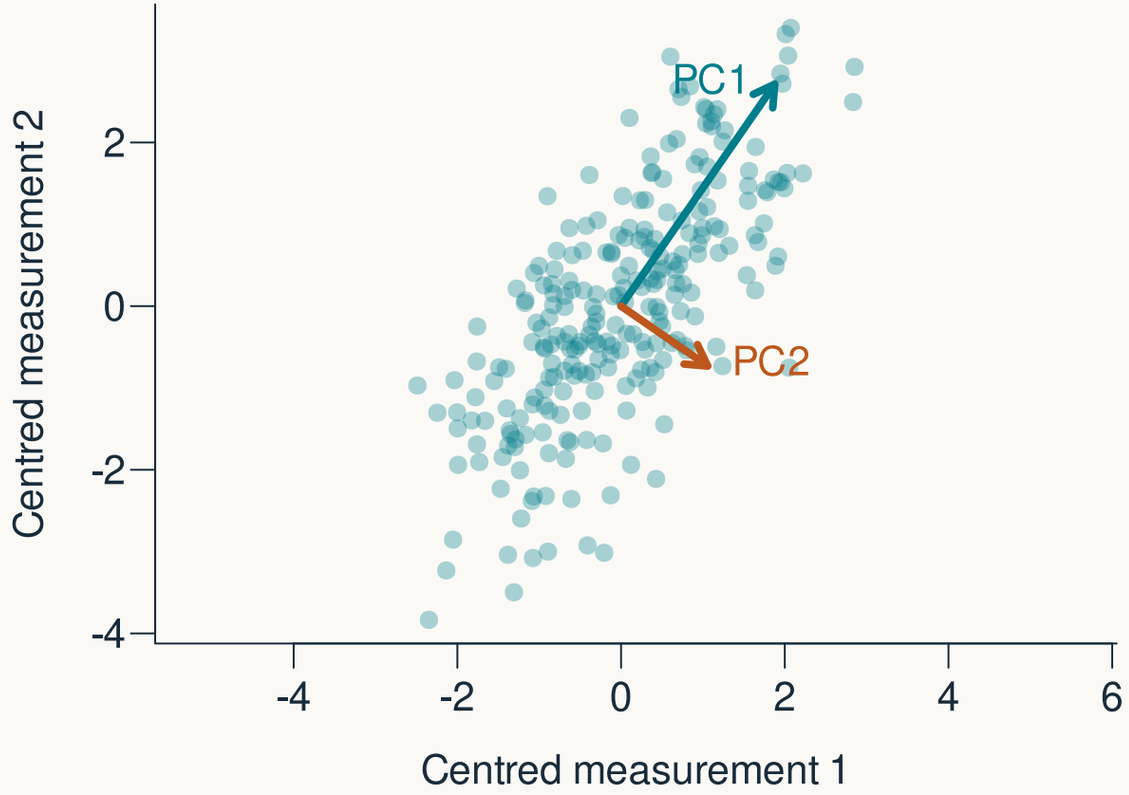
Eigenvectors keep the same line of action

$$\mathbf{S}\mathbf{v} = \lambda\mathbf{v}, \qquad \mathbf{v} \neq \mathbf{0}$$



Teal: input. Orange: transformed vector. λ is the eigenvalue.

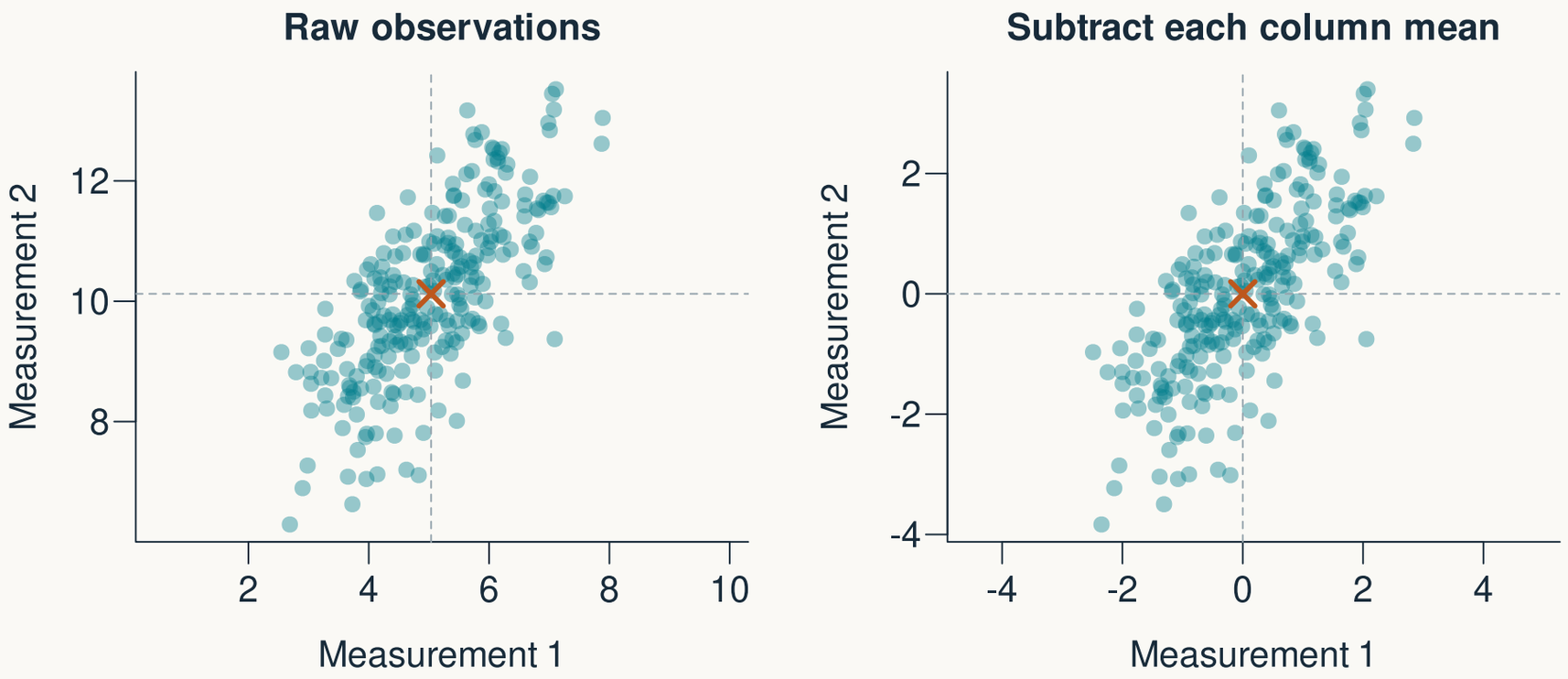
PCA: which direction keeps the most variation?



PC1: largest variance of projected scores.

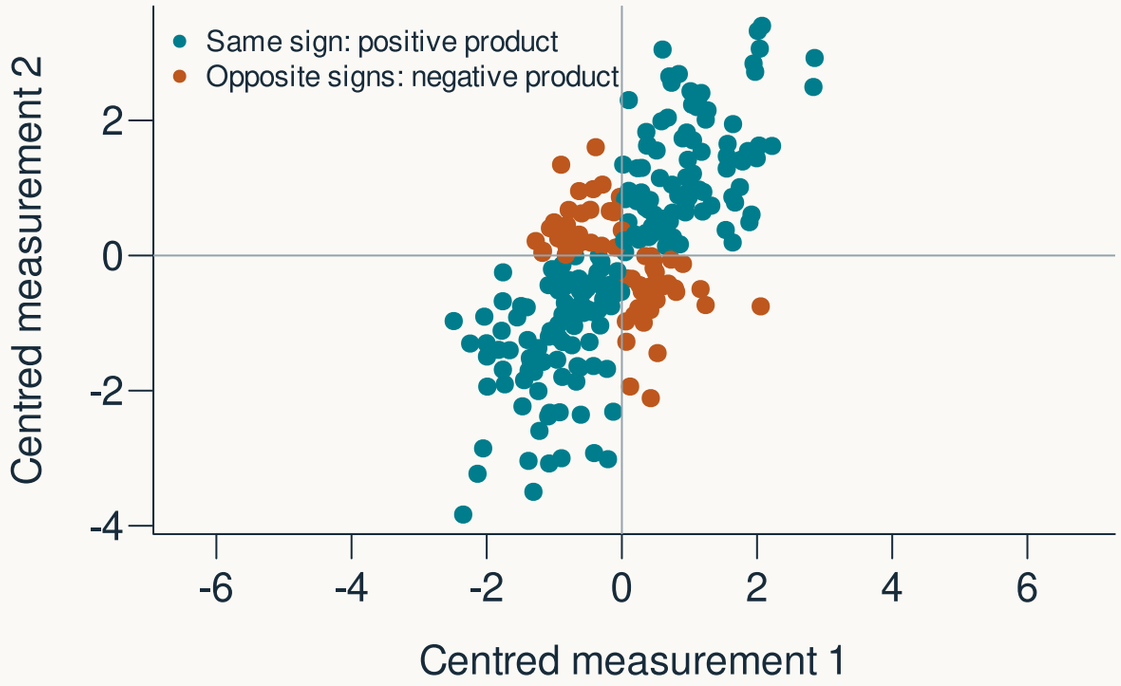
PC2: largest remaining variance, perpendicular to PC1.

Step 1: centre each variable



$$X_{c,ij} = X_{ij} - \overline{X}_j$$

Step 2: summarise how centred variables vary together



$$\mathbf{S} = \frac{\mathbf{X}_c^\top \mathbf{X}_c}{n-1}$$

$$\begin{bmatrix} \mathrm{Var}(x_1) & \mathrm{Cov}(x_1, x_2) \\ \mathrm{Cov}(x_1, x_2) & \mathrm{Var}(x_2) \end{bmatrix}$$

Diagonal: spread.
Off diagonal: variation together.

Step 3: directions and amounts of variation

$$\mathbf{S}\mathbf{v}_j = \lambda_j \mathbf{v}_j$$

PCA object	What it represents
Eigenvector $\mathbf{v}_j$	A unit direction: the **loading vector**
Eigenvalue λ_j	Variance of scores along that direction
Largest eigenvalue	The first principal component direction

R: build the example and find the directions

```r
# simulate correlated data
n <- 250
x1 <- rnorm(n)
x2 <- x1 + rnorm(n)
X <- cbind(x1 + 5, x2 + 10)

# center the variables
Xc <- X
for (j in 1:ncol(X)){
  Xc[, j] <- X[, j] - mean(X[, j])
}

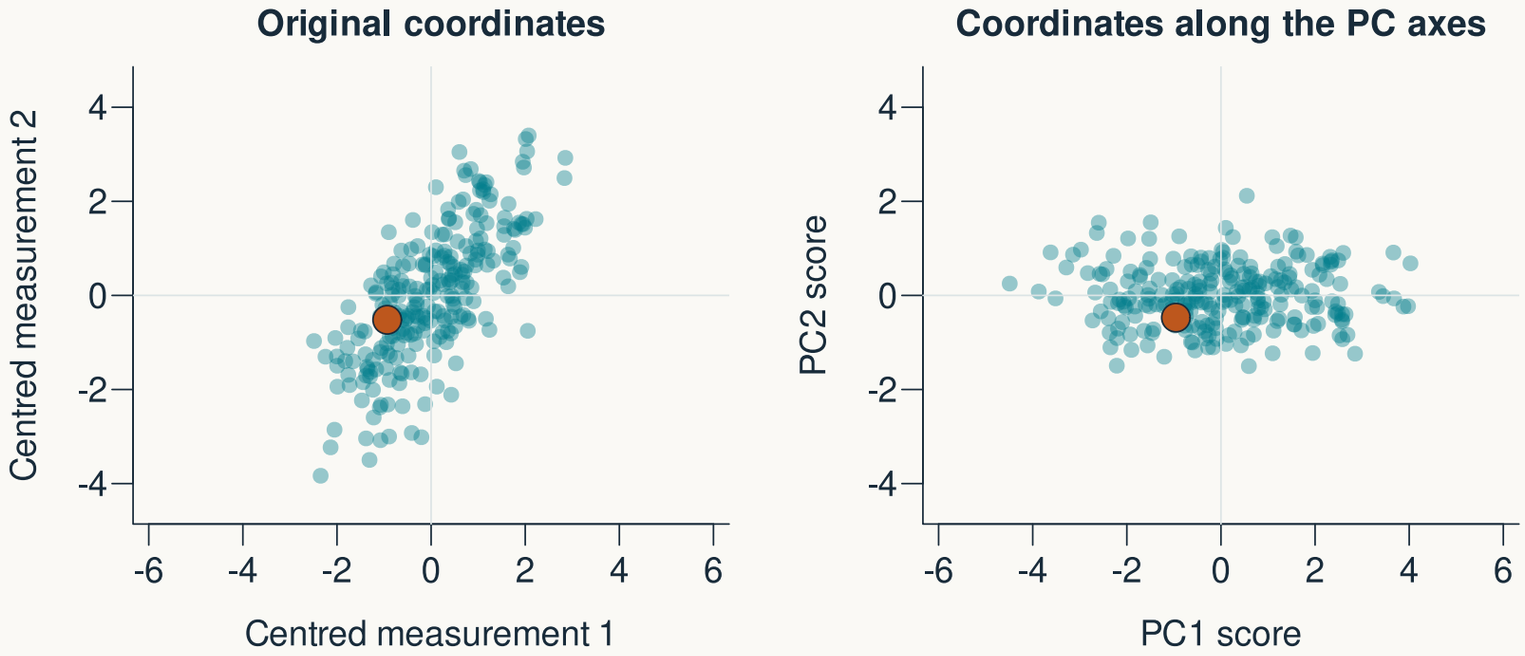
# compute variance-covariance matrix
S <- t(Xc) %*% Xc / (n - 1)

# computes eigenvalues and eigenvector
decomposition <- eigen(S, symmetric = TRUE)
V <- decomposition$vectors
lambda <- decomposition$values
```

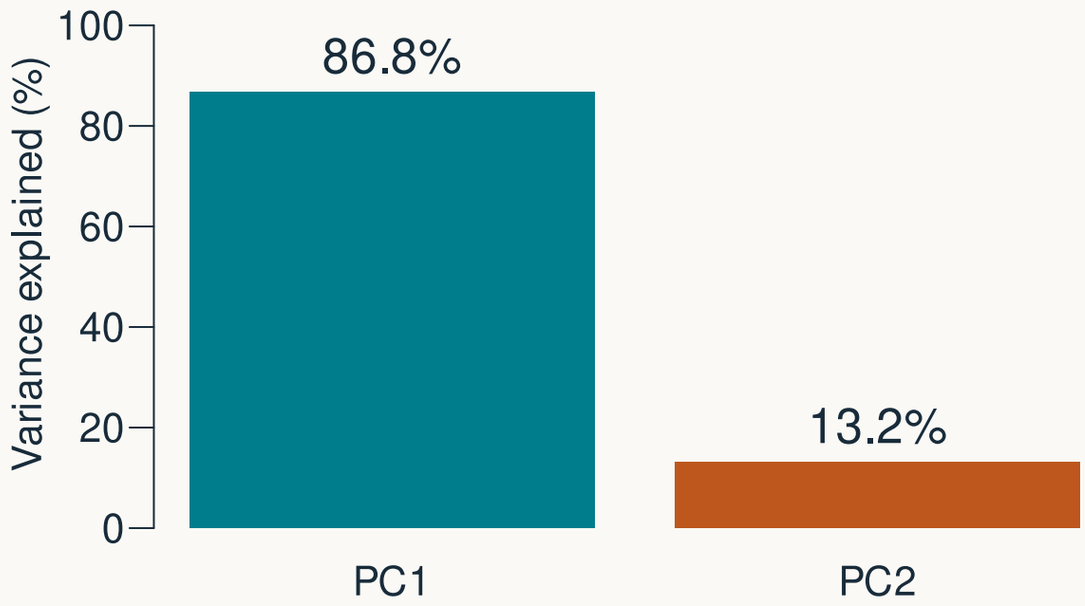
Complete, commented example: pca_example.R

Step 4: scores are the new coordinates

$$\underbrace{\mathbf{Z}}_{n\times p} = \underbrace{\mathbf{X}_c}_{n\times p}\underbrace{\mathbf{V}}_{p\times p}$$



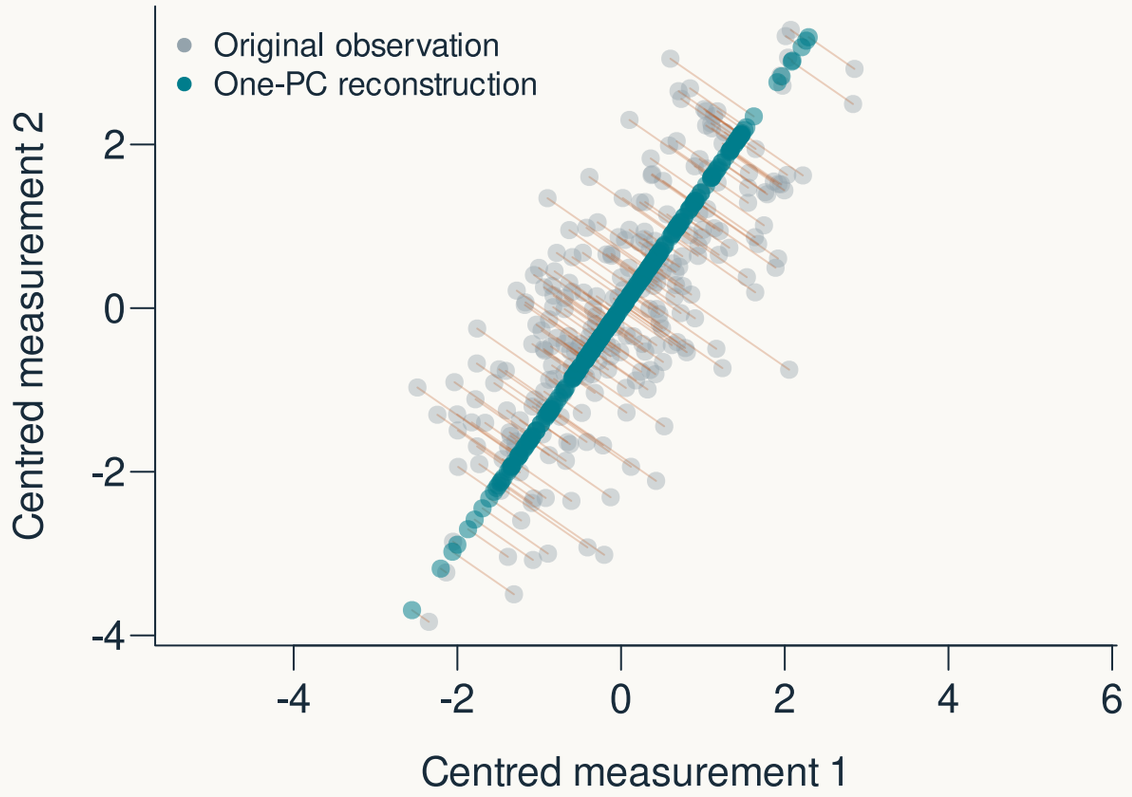
How much variation does each component retain?



$$\text{fraction for PC } j = \frac{\lambda_j}{\sum_k \lambda_k}$$

PC1 retains **86.8%** here.

Step 5: keep one score and reconstruct an approximation



$$\hat{\mathbf{x}}_c = z_1 \mathbf{v}_1$$

One coordinate gives a point on a line.

Add the means back to recover the original units and location.

R: project, measure retained variance, reconstruct

```r
# compute scores (coordinates along PC axes)
scores <- Xc %*% V

# variance explained by each PC
lambda <- decomposition$values
variance_fraction <- lambda / sum(lambda)

# select first PC scores
z1 <- scores[, 1, drop = FALSE]
v1 <- V[, 1, drop = FALSE] # 1st eigenvector

# keep 1PC and do approximate reconstruction
Xc_approx <- z1 %*% t(v1)
X_approx <- Xc_approx
for (j in 1:ncol(X)) {
  X_approx[, j] <- Xc_approx[, j] + mean(X[, j])
}
```

Which changes should affect PCA?

A. Add 100 to every value in one column.

B. Multiply one column by 100.

C. Reverse a loading vector and its corresponding scores.

A: centring removes it. **B:** generally changes covariance PCA.
C: same axis and reconstruction; opposite signs.

PCA in a paper: a trajectory through neural population activity

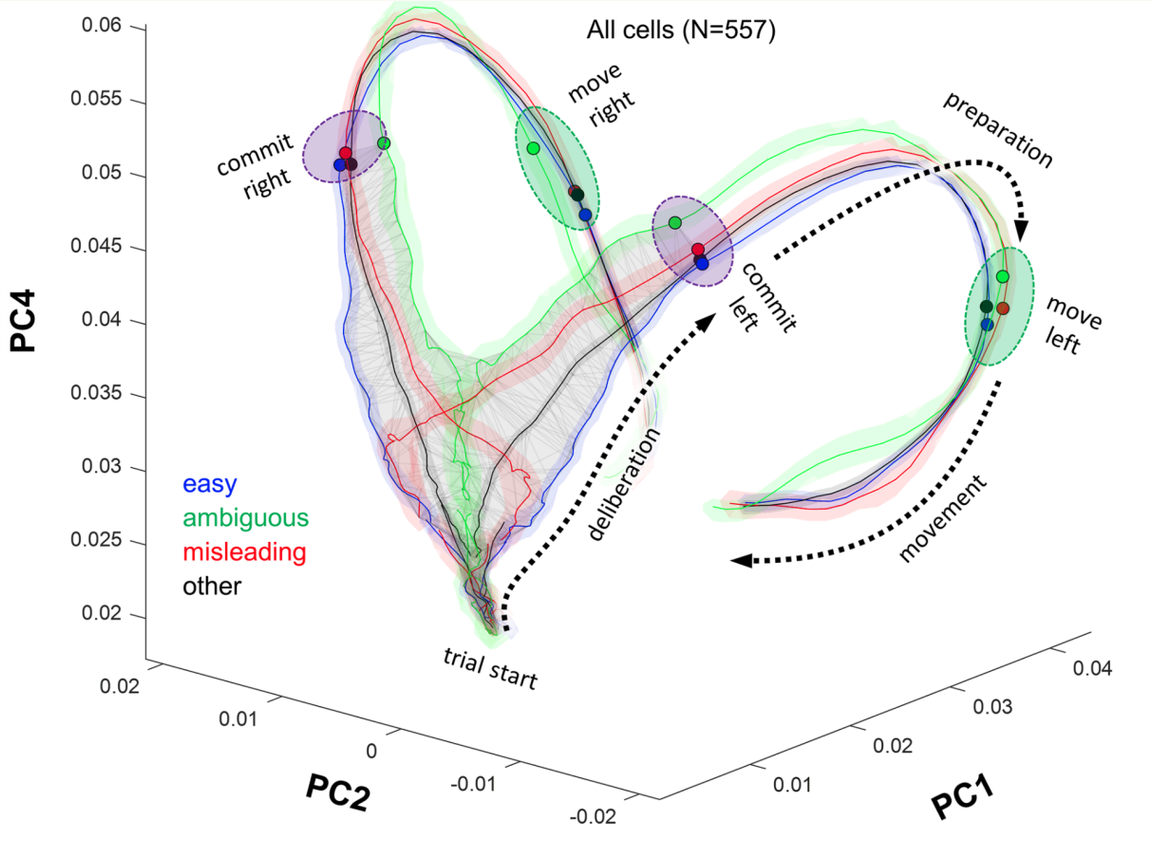


Figure 4A

Each location = population activity at a time point.

PC axes = weighted combinations of neural responses.

The line links successive states.

Thura, Cabana, Feghaly & Cisek (2022), *PLOS Biology*, Figure 4A. doi:10.1371/journal.pbio.3001861.

Part 2 summary

Vectors represent points/directions in space; **matrices** can be seen as functions that transform them.

Linear combinations of vectors can create new vectors from old ones; **span** is the space of possible combinations; **linear (in)dependence** tells us when vector carry redundant information.

Matrix–vector multiplication is a linear combination of the matrix columns; **dot products** connect algebra and geometry (gives angular distances between vector orientations); and **matrix multiplication** can be seens as “combination of combinations” of columns or a sequence of dot products.

Eigenvectors are vectors which are fixed in direction under a given linear transformation (a matrix); the scaling factor of these eigenvectors is then called the **eigenvalue**. (Roughly the eigenvalue is a measure of the distortion induced by the transformation and the eigenvectors tell you about how the distortion is oriented.)

PCA uses eigen-decomposition to find directions of maximal variance for constructing a simpler, interpretable representations.

Read the mathematics in a paper

See this…	Read it as…
$\mathbf{A}\mathbf{x}$	A combination of columns; a transformation
$\mathbf{a}^\top\mathbf{b}$	Multiply corresponding entries, then add
$\mathbf{S}\mathbf{v} = \lambda\mathbf{v}$	A special direction and its scale factor
$\mathbf{Z} = \mathbf{X}_c\mathbf{V}$	Centred observations in new coordinates

> Quiz: how do **loadings**, **scores** and **eigenvalues** differ?

OPTIONAL /

Identity and inverse: leave unchanged, or undo

$$\mathbf{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \mathbf{I}\mathbf{x} = \mathbf{x}$$

$$\mathbf{S}^{-1} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1 \end{bmatrix} \qquad \mathbf{S}^{-1}\mathbf{S} = \mathbf{I}$$

> An ordinary inverse exists only for a **square, full-rank** matrix.

OPTIONAL /

A linear model is a matrix-vector product

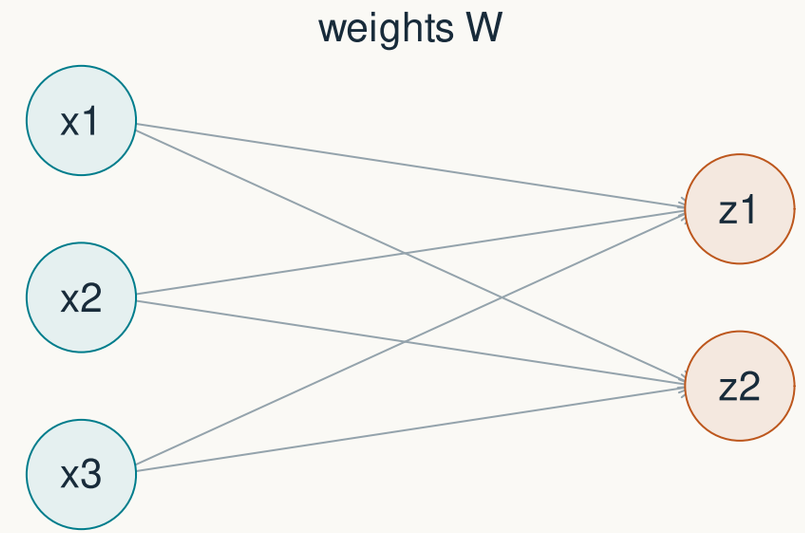
$$\hat{\mathbf{y}} = \underbrace{\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} 5 \\ 2 \end{bmatrix}}_{\beta} = \begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$$

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

Rows = observations. Columns = intercept and predictor. Weights = coefficients.

OPTIONAL /

A network layer combines inputs using weights



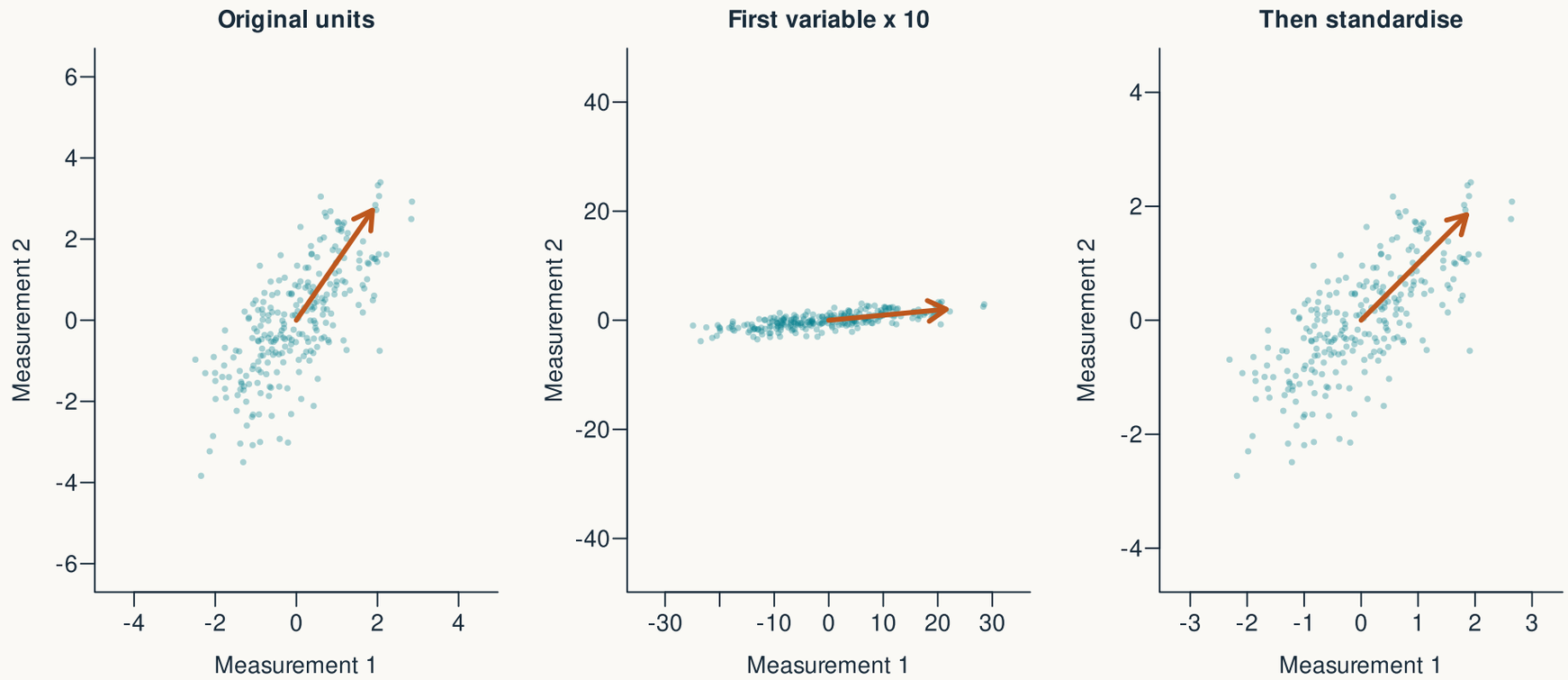
$$\underbrace{\mathbf{z}}_{2\times1} = \underbrace{\mathbf{W}}_{2\times3}\underbrace{\mathbf{x}}_{3\times1} + \underbrace{\mathbf{b}}_{2\times1}$$

$$\mathbf{y} = g(\mathbf{z})$$

Weights → sums → activation.

OPTIONAL /

Scaling changes the question PCA answers



Centring removes offsets. **Standardising** also divides by each variable's SD.